

Build an AI-Assisted Autonomous Futures Trading System

A complete beginner's step-by-step guide — from zero trading and zero AI experience to a live, self-funding, ever-improving trading desk.

The one law this whole design obeys

AI DISCOVERS. HUMANS SIGN. FROZEN RULES TRADE.

Your research AI (Hermes) hunts for improvements around the clock. A human approves every change with one signature. The program that touches real money is a frozen, certified rulebook with no AI inside it. This is what makes the system both ever-improving AND provably safe — remove any leg and you lose one or the other.

Who does what — the three intelligences

HERMES (third-party AI) — the tireless research analyst. Reads every journal and study, hunts edges, drafts proposals. Its long-term memory is an Obsidian vault.

A CODING AI (e.g. Claude Code) — your builder. You describe what a layer must do; it writes, tests, and fixes the software. You never need to write code yourself.

YOU + A RISK PARTNER — the signatures. You own the machine and confirm the checks; your partner owns the risk numbers and the arming key. Neither can launch alone.

Honest warning

Futures trading can lose money quickly. Funded-account evals limit your downside to the eval fee, but nothing here is financial advice. Build the safety layers FIRST, test everything on simulated accounts, and never skip a gate because you're excited.

Version 1.0 — July 2026. Companion to the working system documented in the ai-trading-agents repository.

The map — ten phases, three big stages

Everything below happens in order. Phases 1-4 need no money at risk and no special accounts (a laptop is enough). Phases 5-7 spend a few hundred dollars setting up infrastructure. Phases 8-10 take you live and keep you improving.

Stage	Phase	What you end up with
A. THE LAB (laptop only)	1. Workshop setup	Git repo + Obsidian vault + your two AIs connected
	2. Historical data	Two years of market history on disk
	3. The backtest engine	A simulator that replays history exactly
	4. Strategy discovery	A frozen rulebook + its certified track record
B. THE MACHINE (a few \$100)	5. Accounts	Funded eval + data feed + VPS + chart software
	6. The live bot	The rulebook wired to real-time data and orders
	7. Safety & comms	Halts, kill switches, alerts, daily automations
C. LIVE (real stakes)	8. Deploy & certify	Your VPS provably runs the approved strategy
	9. The proof gates	Every failure mode tested on purpose, then armed
	10. Operate & improve	Autonomous trading + the weekly improvement loop

Budget summary (monthly, one funded account)

Item	Purpose	~Cost/mo
Lucid Trading Flex 50k eval	The capital (until passed; then funded)	\$100-165
ChartVPS (Chicago, Windows)	The always-on machine near the exchange	\$80
Sierra Chart trading package	Data recorder + order gateway	\$36-56
Backblaze B2	Permanent archive of market recordings	\$5
Databento	Historical data for research (one-off buys)	varies
Hermes (third-party AI)	The research analyst	per your plan
Coding AI (e.g. Claude)	The builder/maintainer	per your plan
Obsidian	The shared memory (vault)	free
Telegram, Git/GitHub, Python	Comms, versioning, runtime	free

Phase 1 — Set up the workshop (day 1)

Goal: one folder that is simultaneously your code repository, your AI's memory, and your notebook — with both AIs able to read and write it.

1. Create a free GitHub account and one private repository (e.g. **ai-trading-agents**). Everything — code, research notes, configuration — will live here, with full history.
2. Install Git and Python (3.11+) on your laptop. Both are free; the installers take minutes.
3. Install **Obsidian** (free) and open the repository's **docs/** folder as a vault. Every research note becomes a linked, searchable card; install the community plugin **obsidian-git** so the vault syncs through GitHub automatically.
4. Connect your **coding AI** (e.g. Claude Code) to the repository. Its job from now on: you describe behaviour, it writes code AND tests proving the code works.
5. Connect **Hermes** (your third-party AI) with three abilities: read/write files in the repository (its memory IS the vault), run Python scripts (to execute studies), and post to Telegram (to reach you). How you grant these depends on the Hermes platform - file/shell access on the machine it runs on is the simplest arrangement.
6. Agree the note conventions that make the vault a real memory: one file per finding (FINDING-*.md), one per decision (RULING-*.md), links between them, every number carries its provenance (which study, which data, which date), and superseded notes are marked SUPERSEDED - never deleted. Dead ends are memory too.

You'll know it worked when: you can edit a note in Obsidian, and both AIs can quote it back to you; a test file your coding AI wrote runs green with one command.

Why a vault instead of a database?

Plain markdown files are readable by humans, editable in Obsidian, versioned by git, and consumable by ANY AI - no translation layer, no lock-in to one AI vendor. When Hermes proposes something, the evidence trail is a chain of links a human can walk.

Phase 2 — Buy the history (week 1)

Goal: about two years of the instrument you'll trade (this guide assumes NQ — Nasdaq-100 futures), detailed enough to rebuild what a trader would have seen.

1. Open a **Databento** account (pay per download). Buy for your instrument: 1-minute OHLCV bars (the skeleton), and order-flow data (trades with aggressor side, and market depth if budget allows). Start with bars + trades; depth can come later.
2. Have your coding AI store everything as **parquet files** in a **data/** folder, with a loader script and a test asserting row counts and date ranges never silently change.
3. Ask it to derive the working tables: per-minute 'footprint' (volume, buy/sell delta, average price per minute) and session structure (overnight ranges, session opens).

You'll know it worked when: one command prints 'X bars from DATE to DATE' and a spot-check of a random day matches a public chart of that day.

Phase 3 — Build the time machine (weeks 1-3)

The backtest engine is a simulator that replays history candle by candle and executes imaginary orders under strict rules. It is the single most important piece of software in the project - direct your coding AI to build it with these NON-NEGOTIABLES:

1. **No look-ahead, ever.** When simulating 9:31, the code may only see data up to 9:31. Every feature gets a test proving it. (One look-ahead bug in our reference build inflated results by \$3,500 and took a forensic hunt to find.)

2. **One engine for backtest AND live.** The same function that simulates stop moves and partial exits in research must later drive the live exits. Two implementations WILL disagree - our reference project measured a 'faithful' rewrite agreeing on only 14% of trades.
3. **Deterministic.** Same inputs, same outputs, forever. No randomness, no clock reads.
4. **Tested.** Hundreds of small tests (fills, stops, cancels, session edges). Your coding AI writes them as it goes; you just require they exist and pass.
5. Build in this order: trigger detection (what counts as a candidate setup) - entry/fill simulation (limit orders, cancel-if-price-runs rules) - exit management (stop moves, partial profits, end-of-day flatten) - a per-trade result table.

You'll know it worked when: replaying the same month twice gives byte-identical trade lists, and the test suite (aim for 300+) is green.

Phase 4 — Find the strategy, with Hermes (weeks 3-8)

This is a LOOP, not a step - and it's where Hermes earns its keep. You are not inventing a strategy from genius; you are mining history for behaviours that repeat, then trying to destroy every candidate until only the robust survive.

1. **Generate candidates.** Hermes fans out study after study over the trade tables: does a wall of resting orders ahead of the trade help? Does time-of-day matter? Does the tape's aggression predict follow-through? Each study is a script + a FINDING note.
2. **Adversarial verification - the sacred step.** For every promising finding, a second pass tries to KILL it: does it hold in a year it never saw? In both half-periods? Is it one lucky trade in disguise? (Our reference desk retracted a '+38R improvement' when autopsy showed 80% of it was a single degenerate fill.) Most candidates die here. That is the system working, not failing.
3. **Freeze survivors.** Winning thresholds become constants in a config file with provenance comments. Live code NEVER re-computes them from data.
4. **Assemble the rulebook** ('the canon'): hard gates (e.g. stop distance bounds) - binary checks - a score - a size ladder (score 3 = half size, 5 = max) - vetoes (news blackout, dead time-zones) - escalations (second trade of the day needs a higher score; shrink after losses).
5. **Run the full book** over both years. The headline (ours: +\$55,989.81 over 383 trades, 13/13 green months) becomes THE REFERENCE NUMBER - the exact result the live system must be able to reproduce to the cent, forever.
6. **Write the risk rules with math, not vibes.** Monte-Carlo the funded account's rules (trailing drawdown, payout cycles) to choose the daily loss halt and the drawdown taper. Your risk partner signs these two numbers personally.

You'll know it worked when: a single command regenerates the reference number exactly, and every rule in the book traces to a FINDING note that survived adversarial review.

Phase 5 — Open the accounts (week 8, ~1 day of clicking)

1. **Lucid Trading - Flex 50k evaluation.** At 'Select Platform' choose the **Rithmic** data feed (NOT CQG - Sierra Chart requires Rithmic). ~\$100-165/mo until passed. If the bot is broken you lose the eval fee, not a bankroll - that asymmetry is why we launch on the eval with no paper-trading period.
2. **R|Trader Pro** (free): log in once and sign the CME non-professional data agreement. Verify your plan includes full 10-level market depth - several strategy checks die without the ladder.
3. **ChartVPS 'Alpha Mark-2'** (or similar): Windows Server, ~\$80/mo, **Chicago** location - the same metro as the exchange servers.
4. **Sierra Chart** trading package (~\$36-56/mo) installed on the VPS: connects to Rithmic, records every tick (.scid file) and the order book (MarketDepthData/*.depth), and exposes the DTC server - the local 'order door' your bot will use.
5. In Sierra: open your contract's chart and set, on the symbol pattern, 'Record Market Depth Data' = Yes and 'Depth Levels to Subscribe' = 10; set the intraday file flush to 1000 ms. Leave Sierra and this chart open ALWAYS - closed chart = blind bot.
6. **Backblaze B2** (~\$5/mo): create a bucket for nightly archives - Sierra deletes depth recordings after ~30 days and they can never be re-downloaded.

You'll know it worked when: Sierra shows live prices on the VPS, and the two recording files grow during market hours.

Phase 6 — Build the live bot (weeks 8-12)

The live bot is the rulebook re-plumbed for real time. Direct your coding AI layer by layer; every layer gets tests against a FAKE broker before it ever meets the real one.

1. **Data tail:** read Sierra's own recording files once per second (simple, license-clean, replayable). Parse the binary formats and PIN them with a checker script - expect the real machine to differ from documentation (ours had a different message numbering AND prices scaled 100x - both caught by the pin checks, both would have silently poisoned everything).
2. **Live features:** the same ~40 observations, computed incrementally, decimal-checked against the research tables.
3. **Live scorer:** the rulebook restated one-trade-at-a-time, proven to reproduce the book row-for-row (383/383 in our build).
4. **Order path:** a DTC client speaking to Sierra: bracket entry (entry + protective stop as one unit), cancel, modify-stop-in-place, partial close, flatten. Golden rule: **read back, never trust** - after every order, ask the server what it actually did.
5. **Managed exits:** the engine itself decides stop moves/partials live (same code as backtest); an executor translates decisions into orders; any doubt = flatten.
6. **The safety spine** - a bodyguard that can only say NO - wraps everything: daily loss halt, drawdown taper to zero, stale-data/spread/rate/duplicate blocks, protective-stop verification, and the ARMING LOCK: without your partner's recorded authorization the order door does not open, structurally.

You'll know it worked when: the full test suite (ours: 650+) runs green, including simulated broker failures, and a 'shadow mode' run journals decisions while provably unable to place an order.

Phase 7 — Mouth, leash, and reflexes (week 12)

1. **Telegram bot** (free, via @BotFather): alerts for every decision, trade, and halt into a group with you AND your risk partner.
2. **Kill switch**: both phones' Telegram IDs are allow-listed; /kill from either writes a kill file that halts the spine. Run the kill listener as its OWN process - a wedged bot must not take its off-switch down with it.
3. **News sentinel**: a small daily script fetches the economic calendar before the session; FAIL-CLOSED rule: no calendar today = no pre-market trading today.
4. **Daily automations** (Windows Task Scheduler): the sentinel each pre-dawn, the Backblaze archive each evening after the close.

You'll know it worked when: a test message buzzes both phones; /kill from each phone halts a running shadow bot; the two scheduled tasks show SUCCESS.

Phase 8 — Deploy to the VPS and certify (launch week)

1. On the VPS: install Python + Git, clone the repository, install requirements, run the FULL test suite. Fix anything red before proceeding.
2. Run the format pin-checks against the real Sierra files. **Expect real discoveries** - in one Sunday our reference build caught three: a message-numbering mismatch, 100x price scaling, and a file-naming quirk that silently skipped the archive's most irreplaceable files. This is why these checks exist.
3. Replay a full recorded day through the live wiring: real bytes - bars - order book - feature rows. Sanity-check prices and the 10/10 book levels.
4. **CERTIFY**: regenerate the strategy book on the VPS. It must print the reference number TO THE CENT, and the replay harness must match the reference trade list exactly (383/383 in ours). One cent off = stop and diff; something drifted.
5. Measure the data-file write delay against a 1-minute decision cycle (ours: ~1-2s median - fine). Record the number.

You'll know it worked when: the certification prints your reference number exactly, on the machine that will trade it, at the exact code version you will arm.

Phase 9 — The proof gates, then arming (launch night)

Deliberately break everything BEFORE real money. Cause each failure; watching and hoping is not evidence. All on Sierra's built-in SIM account (scripts should REFUSE to run order tests on any non-SIM account).

1. **Order surface**: place a far-from-market bracket - verify TWO order IDs come back (entry + protective stop), modify it, cancel it, verify both legs dead. Then a real SIM fill: stop moved in place, partial close, flatten to zero.
2. **Safety spine force-test**: trip every halt rule on the live setup and watch each fire and alert.
3. **Break-it drills**: kill the bot mid-position (must FLATTEN, not ride on the resting stop); cut the data feed (must halt); restart mid-session (no duplicate or lost orders); force one signal through each strategy path.
4. **Dual kill drill**: you AND your partner each /kill from your own phones, verified and journaled.
5. Run one full session DISARMED (shadow) overnight: journals clean, every alert arrives.
6. **The signatures**: you write the confirmation - every gate checked against tonight's actual outputs, the certification number quoted verbatim. Your partner replies with the ARMING TOKEN naming the exact code version. Config points at the funded account. Armed.
7. **The discipline that makes it safe**: any red gate = launch next session instead. 'The answer is Tuesday, not arm anyway.'

You'll know it worked when: the bot takes its first live trade with no human touching anything - and both of you watching the alerts channel anyway.

Phase 10 — Operate, and improve forever (the Hermes flywheel)

The bot now trades autonomously: protective stop on every entry, halts that stop the day on damage, a taper that shrinks size near the account floor, fail-closed reflexes for every fault. Your ongoing jobs are watching alerts (first weeks), one supervised morning per quarterly contract roll, and turning the improvement crank:

1. **Nightly - Hermes audits:** reads the day's journals, diffs live behaviour against the book (the certification re-checked daily), flags anomalies to Telegram.
2. **Weekly - Hermes researches:** re-runs the study library over the growing live record; hunts decay in old edges and candidates for new ones; every finding faces the same adversarial gauntlet as Phase 4.
3. **Survivors become memos** in the vault: evidence, proposed frozen constant, expected delta to the reference number, out-of-sample plan.
4. **You and your partner sign** (or reject) each memo - one click each.
5. **The mechanical ship:** constant freezes - book regenerates - VPS re-certifies to the cent - new reference number - the better bot trades tomorrow. (Our reference desk ran this loop five times in a single day; one proposal was retracted by its own autopsy. Both outcomes are the loop working.)
6. **Scale** (optional): more funded accounts running the same certified brain, each sized by its own account state.

Why not let Hermes change the live rules directly?

Because certification dies the moment rules drift: today's 'provably the approved strategy' becomes 'whatever it mutated into overnight'. Because a learner updating on a few dozen live trades mostly learns noise (remember the retracted +38R). And because every safety gate above assumes the thing being gated holds still. The human signature costs one click and buys an audit trail for every dollar. Keep the click.

Appendix — the one-page master checklist

#	Milestone	Proof
1	Workshop: repo + vault + both AIs connected	AIs quote a vault note; a test runs green
2	History bought and loaded	Loader prints expected ranges; spot-check matches a chart
3	Backtest engine, no look-ahead, tested	Byte-identical replays; 300+ tests green
4	Strategy found, adversarially verified, frozen	Every rule traces to a surviving FINDING note
5	Reference number locked	One command reproduces it to the cent
6	Risk numbers Monte-Carloed and signed	Partner's written sign-off in the vault
7	Accounts: eval (Rithmic) + VPS + Sierra + B2	Live prices on the VPS; recording files growing
8	Live bot built against a fake broker	Full suite green incl. failure injections
9	Safety spine + arming lock	Shadow mode provably cannot place an order
10	Telegram + dual kill + sentinel + archive	Both phones buzz; /kill halts; tasks registered
11	VPS deploy: suite green, formats pinned	Pin checks PASS on real files (bugs found = good)
12	CERTIFIED on the VPS	Reference number to the cent; replay matches exactly
13	SIM order surface proven	Two IDs back; modify/cancel/partial/flatten verified
14	Break-it drills all pass	Mid-position kill flattens; feed cut halts; restart clean
15	Written confirmation + arming token	Token names the exact code version
16	LIVE	First autonomous trade, both humans watching alerts
17	Flywheel running	Nightly audits; signed memos shipping weekly

Glossary (the six terms that matter)

Term	Plain meaning
Backtest	Replaying history through your exact rules to see what they would have done
Frozen constant	A number measured once, written down with provenance, never recomputed live
Certification	Proving the live machine reproduces the approved track record to the cent
Fail-closed	When anything is uncertain, stop/flatten - never guess and continue
Read-back	After telling the broker anything, ask it what it actually did before trusting
Arming token	The recorded human authorization, naming an exact code version, that opens the order door